

# Credit Assignment for LLM Agents under Context Truncation

Sean Conlon

MSc thesis, Bocconi University, advised by Martino Banchio. September 2026.

Code: [github.com/sbconlon/prime-rl](https://github.com/sbconlon/prime-rl) [github.com/sbconlon/verifiers](https://github.com/sbconlon/verifiers)

## Summary

LLM agents on long tasks run out of context. The standard fix is to drop the oldest turns, making the task partially observable. I trained GRPO on ALFWorld with and without truncation. With a 2k token window the policy solves 56% of tasks. With a full window it solves 94%, and the gap grows with task length. I then implemented ARM, a credit assignment method adapted from counterfactual regret minimization, inside Prime Intellect’s prime-rl. ARM replaces GRPO’s single per-trajectory advantage with a per-decision regret computed by learned value networks. It runs end to end. It never sustains a learning signal. Every symptom traces to one network, the regret network, which never learns to tell admissible actions apart. Two stabilizers narrowed the cause to a gap in the data it sees. The rest of this document covers why ARM should help, what the failure looks like, and what I think fixes it.

## 1. Long-horizon agents and credit assignment

An agent task is a sequence of decisions with a reward at the end. ALFWorld is a text-based household environment. The agent is given a goal, put a clean mug on the countertop, and issues actions until it succeeds or runs out of turns. Good solutions take three to twelve actions, and the agent only learns whether it succeeded when the episode ends.

The reward is one number for the whole episode. Credit assignment turns it into a grade for each decision, based on how much that decision contributed to the outcome.

GRPO doesn’t grade decisions at all. It samples a group of rollouts for the same prompt, normalizes their rewards within the group, and broadcasts each rollout’s normalized reward across every token in it. Every decision in a successful rollout is rewarded equally, including the mistakes. Every decision in a failed one is punished equally, including the good moves.

On a three-step task, uniform credit is close to correct. With so few decisions, each one has to matter. On a twelve-step task the importance is spread thinner, so more of the credit lands on decisions that didn’t matter and the signal for the ones that did is buried.

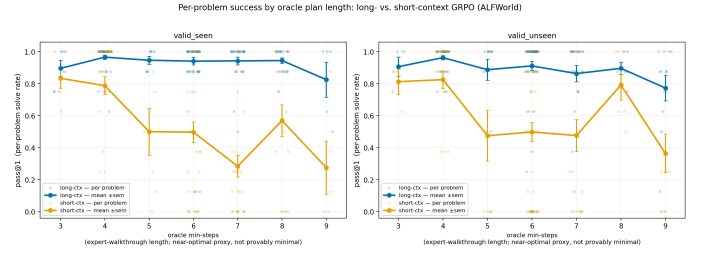
## 2. Truncation as partial observability

Long tasks present another problem. The context window is fixed and an episode’s history isn’t. Once the history overflows, the oldest turns are dropped. On ALFWorld with a 2k window, the first eviction fires around turn four or five. From then on the agent can’t see the task description or its earliest observations.

Truncation is an observation function that maps the full history to a shorter one, and it’s lossy. Two episodes that started with different goals, heat a mug and cool a mug, and then followed the same recent turns look identical to a truncated agent. The agent is in a partially observable MDP, and the partial observability was manufactured by the training setup, not the environment. In game theory the same structure is called an information set, a set of states the decision-maker can’t tell apart. That connection is what motivates the method in section 4.

## 3. GRPO under truncation

The setup is two GRPO runs from the same supervised starting point, a Qwen2.5-1.5B model distilled from a 7B teacher on ALFWorld. One is trained and evaluated with a 16k window, which never truncates in practice. The other is trained and evaluated with a 2k window, which drops roughly two thirds of the history by the end of a long game. Same data, same hyperparameters, same evaluation of eight attempts per game on



**Figure 1:** Per-problem pass@1 against oracle plan length, long-context vs. short-context GRPO, seen and unseen splits.

a seen split and an unseen split, scored as pass@1.

Truncation roughly halves accuracy, from 94% to 56% on seen games and 90% to 58% on unseen.

That GRPO does worse with a smaller window is not surprising on its own. The claim is that it does worse because of partial observability. Figure 1 shows this by plotting per-game success against the length of an expert solution to that game.

The full-context policy is flat. It solves nine-step games about as often as three-step games. The truncated policy matches it on games that finish in four or five steps, which is exactly where eviction starts, and then falls from about 80% to about 30% as games get longer. The flat full-context line rules out the alternative that long games are just harder. They’re harder only when the window hides their start.

One objection needs answering. Maybe long games are unsolvable once the opening turns are gone, and no training algorithm could fix that. Two facts say otherwise. First, evicted turns are gone, but the information in them doesn’t have to be. An agent that restates the task in each reasoning block carries it forward long after the original statement is evicted. Second, the truncated policy’s pass@8 is 79% on seen games and 72% on unseen, so given eight tries it solves most of what it fails at pass@1. The policy can do it. It doesn’t do it reliably, and that’s a training signal problem.

## 4. ARM

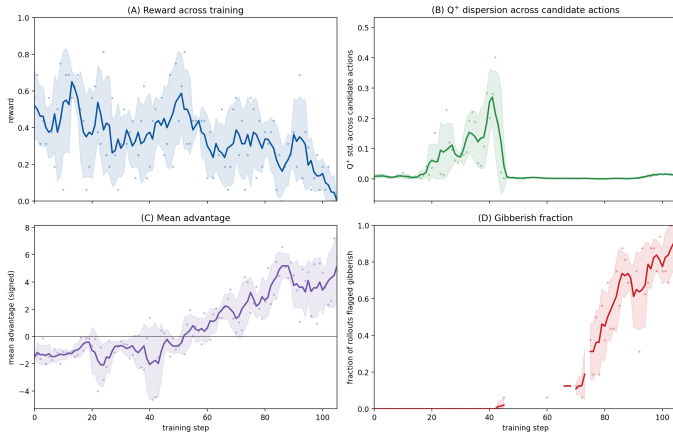
Counterfactual regret minimization is the algorithm behind every strong poker bot. It works on games where players can’t see the full state, and it assigns credit per decision rather than per game. At each information set it tracks, for each action, the cumulative regret, meaning how much better that action would have done than the policy’s average, summed over training iterations. The policy is then set by regret matching, putting probability on each action in proportion to its positive regret. CFR’s regret bound scales with the number of information sets and not with their structure, making it robust to partial observability.

ARM (Jin, Keutzer, and Levine, ICML 2018) is the single-agent version. It replaces the information set with the observation, replaces the tabular regret with a learned function trained on  $n$ -step returns, and sets the policy by regret matching on the result. It matches policy gradient and Q-learning on fully observable tasks and outperforms them on partially observable ones.

ARM-for-LLMs starts from GRPO and swaps the advantage. Nothing else changes. Each decision gets

$$A(a) = \log \pi_{\text{RM}}(a | o) - \log \pi(a | o)$$

where  $\pi_{\text{RM}}$  is the regret-matching distribution over the available actions, computed from a learned cumulative-regret network  $Q^+$  and a learned state-value network  $V$  as  $\pi_{\text{RM}}(a | o) \propto \max(0, Q^+(o, a) - V(o))$ , and  $\pi(a | o)$  is the policy’s own probability of the action. The log-ratio form



**Figure 2:** The cold run. Reward,  $Q^+$  dispersion across candidate actions, signed mean advantage, and gibberish fraction, each against training step.

is forced by the fixed point. It vanishes exactly when the policy equals the regret-matching distribution, so  $\pi_{RM}$  is what the update converges to. The clipped surrogate, the rollout structure, and the training loop remain unchanged from GRPO.

ARM’s regret-matching policy outputs a distribution over environment actions. An LLM outputs a distribution over tokens. The advantage is the log difference between the two, so they have to be distributions over the same thing. The environment action comes at the end of a reasoning block the environment never sees, so the LLM’s probability of an action is a marginal over every reasoning block that could have produced it, and the model doesn’t represent that directly. I estimate it by sampling two reasoning blocks at each decision point, executing the action from one, and scoring that action under the other, a Rao-Blackwellized Monte Carlo estimate with the executed block held out so it doesn’t bias its own estimate upward. The scalar advantage for the executed action is then broadcast across every token of the response, so the reasoning that produced a good action is reinforced along with it.

## 5. What happened

Three runs on ALFWorld under 2k truncation, from the same SFT checkpoint used for the GRPO runs.

The first diverged at step 12. The cumulative regret grew without bound, from 0.30 to 1.37, and the policy collapsed with it. The cumulative regret only ever adds, and without a discount nothing pulls it back down. Discounted CFR (Brown and Sandholm, 2019) applies a discount factor to the carried-forward regret. With that factor at 0.9, the regret stayed bounded in every subsequent run.

The second run is the cold baseline, 106 steps, in Figure 2. Reward oscillates between 0.06 and 0.81 with no trend.  $Q^+$  dispersion across the admissible actions, the direct measure of whether the regret network can rank them, sits near zero for 20 steps, ramps to 0.40 at step 42, and collapses to 0.003 by step 45, where it stays. The value and  $Q^+$  losses fall throughout and don’t move through the collapse. The first gibberish rollout appears at step 44. Entropy climbs from around 0.3 to between 3 and 5 after step 50, and by step 80 most of each batch is rare-token garbage that the gibberish filter removes from training.

The third run tests warm-starting. If  $Q^+$  is failing because it has to learn state values and track a changing policy at once, pretraining it on the state values should leave it only the second job. So I pretrained  $V$  and  $Q^+$  on Monte Carlo returns from the SFT policy and started RL from the warm networks. On the value side it worked: step-0 losses fell from 1.85 and 1.75 to 0.139 and 0.044. Everything else was the same.  $Q^+$  dispersion stayed near 0.005 for 60 steps and rose only slightly after. The first gibberish rollout appeared at step 55, the entropy climbed on the same trajectory, and reward never established a trend.

## 6. Diagnosis

The regret network,  $Q^+$ , never learns to rank the admissible actions at a given state, and everything else follows from that. It can’t rank because its training data never contain a comparison. The pipeline is on-policy and the policy is peaked, so  $Q^+$  sees one action per state and assumes every alternative is worth the same. The warm-start proved it. Pretraining made  $Q^+$  accurate on the actions it had seen and no better at ranking the ones it hadn’t.

A flat  $Q^+$  makes the regret-matching target uniform over the admissible set. The advantage on the sensible actions the policy actually takes is then mildly negative, a gentle push toward uniform. When the regret network has no opinion, uniform exploration is the right answer. In a tabular game with a few hundred states it works. With thousands of games, twelve-step tasks, and a hundred training steps, it’s a random policy that can’t converge in time.

The uniform target would not be disastrous on its own if it didn’t produce an advantage signal strong enough to drive the policy into gibberish. The advantage is  $\log \pi_{RM}(a) - \log \pi(a)$ , and the second term rewards an action in proportion to how unlikely the policy thought it was. As mass leaks toward rarer actions and one of them gets sampled, it collects a large positive advantage for being rare, bounded only by a probability floor of  $10^{-6}$ , which makes it likelier to be sampled again, and that feedback loop runs until the policy has spread across the vocabulary. A token-level policy can only become uniform over actions that way, so the end state is rare-token gibberish. The onset lands at about the same step in both runs because the loop’s speed is set by the advantage and the learning rate, and  $Q^+$  is flat in both.

The cold run takes one extra hit. The first time  $Q^+$  forms opinions, at steps 41 and 42, they contradict the policy. The advantage on the policy’s own actions swings large and negative, the policy destabilizes, and gibberish begins two steps later.

In conclusion, CFR works because it evaluates every action at every information set, and ARM has to learn those counterfactual values from samples. On-policy samples give it one action per state. They could still be enough if the policy explored long enough to cover the alternatives, but the moment it starts exploring, the advantage on the rare actions it tries blows up and takes the policy with it. Either the samples never come, or the run doesn’t survive collecting them.

## 7. What I’d try next

Give  $Q^+$  counterfactual data. Branch from visited states, execute each admissible action, and roll each branch to the end under the SFT policy. That produces a return for every action at a state instead of just one, which is exactly what  $Q^+$  needs in order to rank. It’s computationally expensive, but the samples would be worth it as pretraining data.

Shrink the training set. Fewer games means observations recur, which gives on-policy data a chance to show the same state under different actions.

Bound the rare-action term. The advantage rewards rare actions with no practical bound. Cap it, or fall back to GRPO’s advantage when  $Q^+$  has nothing to say, so an uninformed  $Q^+$  can’t wreck the policy while it learns. This doesn’t teach  $Q^+$  to rank, but it keeps a run alive long enough for the ranking to be learned.

## 8. Code

ARM-for-LLMs is built inside Prime Intellect’s prime-rl as two additional processes, a value-network server and a value-network trainer, alongside vLLM and the policy trainer. The prime-rl fork ([github.com/sbconlon/prime-rl](https://github.com/sbconlon/prime-rl)) has the architecture, the ARM and PPO advantage functions, the warm-start pipeline, and instructions for running a GRPO baseline and an ARM run. The verifiers fork ([github.com/sbconlon/verifiers](https://github.com/sbconlon/verifiers)) has the ALFWorld environment, the context-truncation mechanism, and the reasoning-block sampling and scoring.